Deploying Loan Risk AI Agent on IBM Cloud Code Engine

GitHub repository: https://github.com/IBM/ai-agent-for-loan-risk

Table of Contents

Prerequisites ***1***

Deployment Steps ***2***

1. **Confirm prerequisites and gather information** **2**
2. **Create a Project in watsonx.ai** **2**
3. **Create a namespace in Container Registry** **4**
4. **Deploy application to Code Engine** **5**
5. **Validate the application.** **9**
6. **Enhance application with additional optional features.** **9**

Optional Features ***10***

Prerequisites

Requires IBM Cloud account with:

- watsonx.ai services (refer). This includes:
  - watsonx.ai Runtime
  - watsonx.ai Studio
  - Cloud Object Storage
- Account user with administrative privileges for:
  - watsonx.ai – to create Projects, access services, store assets, etc. (refer)
  - Code Engine – to create Project and deploy the application. (refer1 refer2)
  - Container Registry – to create namespace, store images for deploying the application
- Account user's API key (refer)

Deployment Steps

Note on costs: The deployment will incur costs for Code Engine runtime, watsonx.ai runtime and watsonx.ai LLM inferencing.

1. Confirm prerequisites and gather information

1.1. Note the user's API key value. It will be needed for creating container registry secret. It will also be the environment variable WATSONX_AI_APIKEY.

1.2. Note the region where watsonx.ai is deployed. Get the service URL API endpoint for your region from this list - e.g., for us-south/Dallas region it is https://us-south.ml.cloud.ibm.com. This will be the environment variable WATSONX_SERVICE_URL.

1.3. Note/decide the region to deploy the application resources, e.g. us-south, use-east, eu-de etc.

1.4. Note/decide/create the resource group to use in the account, e.g. agentic-ai-rg

2. Create a Project in watsonx.ai

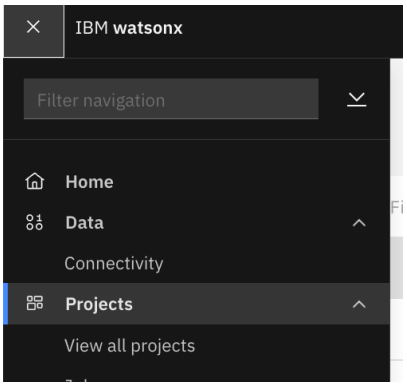
(Or use an existing one and capture its Project Id)

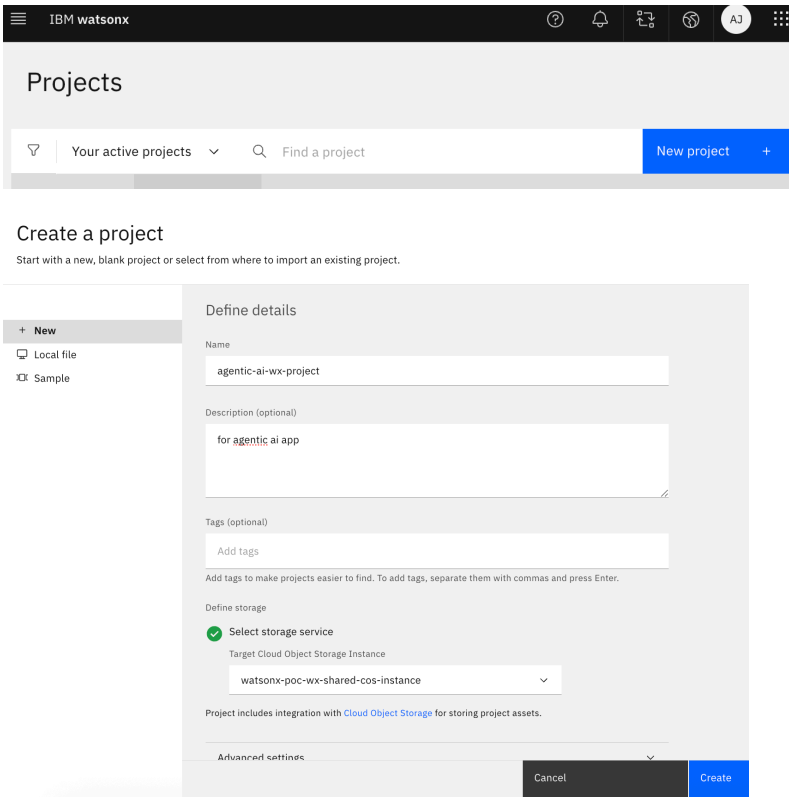
To create a new Project, you will need to configure the Object Storage. Associate the Project with watsonx.ai Runtime service.

Detailed steps for reference:

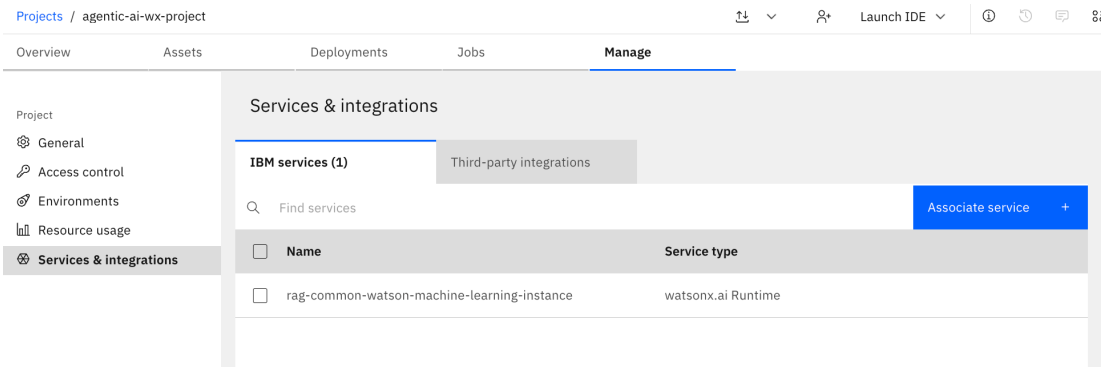
2.1. Create a Project. Also define the Cloud Object Storage for storage.

https://dataplatform.cloud.ibm.com/docs/content/wsj/getting-started/projects.html?context=wx&audience=wdp
https://dataplatform.cloud.ibm.com/projects/?context=wx

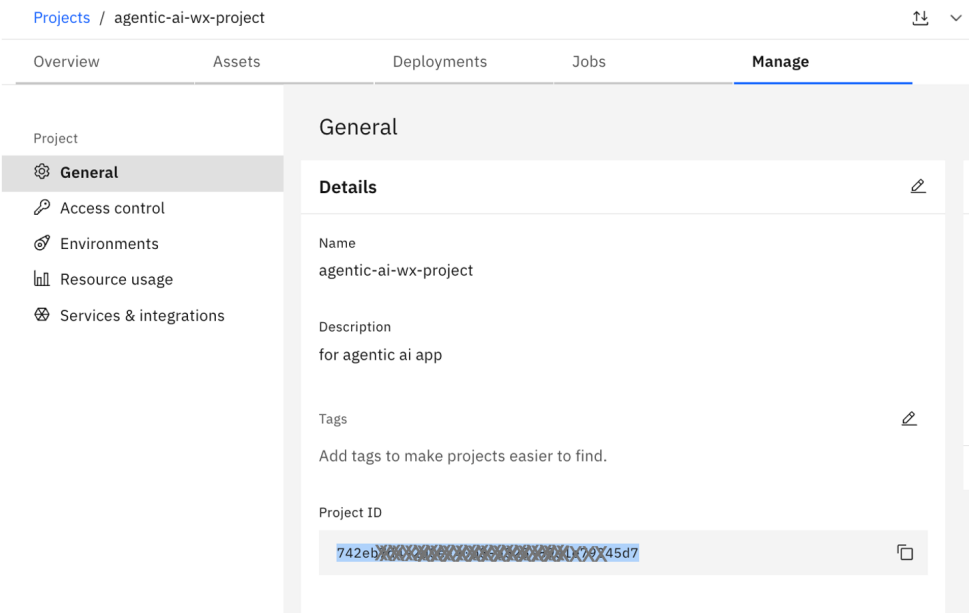




2.2. From the Manage tab > Services and integrations >Associate service, - Associate it with the watsonx.ai Runtime service.



2.3. From the Manage > General tab, capture the Project ID

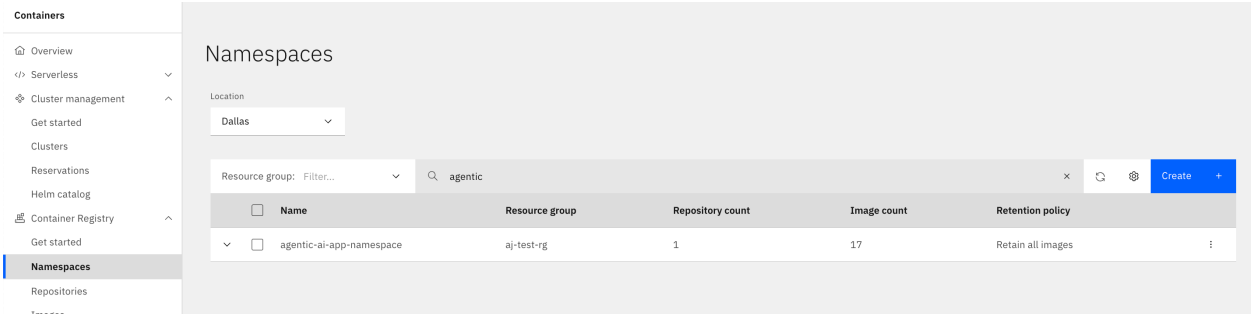


2.4. Note the Project Id. This will be the environment variable WATSONX_PROJECT_ID.

3. Create a namespace in Container Registry

Detailed steps for reference:
https://cloud.ibm.com/containers/registry/namespaces

Name: agentic-ai-app-namespace



4. Deploy application to Code Engine

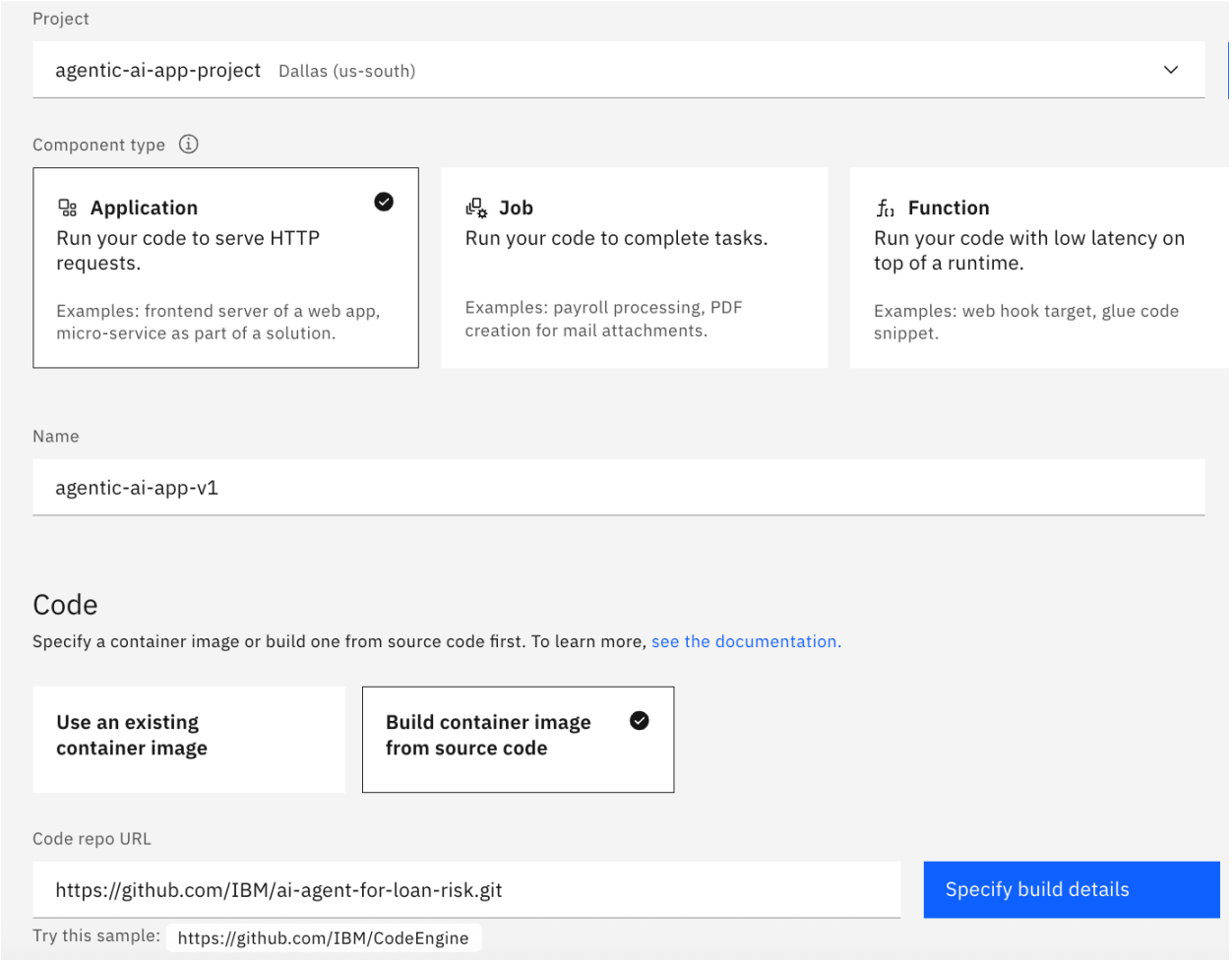
Detailed steps for reference:

Refer section [Deploying your app from repository source code from the console]

Below is some information you will need to complete the steps.

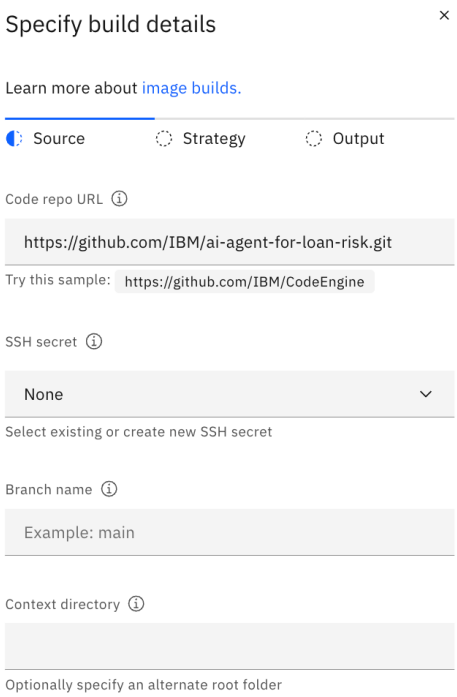
4.1. Click Create Project

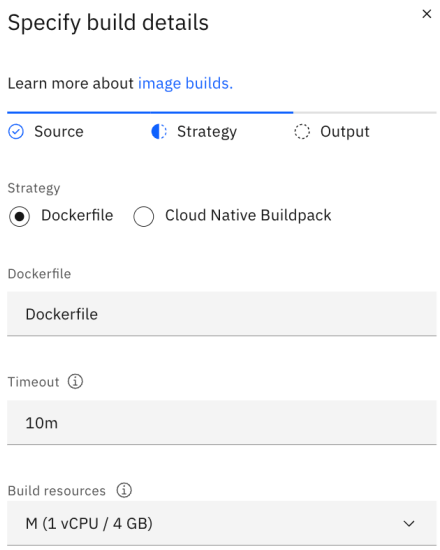
- Project name: agentic-ai-app-project, Select region and resource group.
- Component type. Select Application. Application name: agentic-ai-app
- Code. Select Build Container image from source.
- Code repo URL: https://github.com/IBM/ai-agent-for-loan-risk.git



4.2. Specify build details.

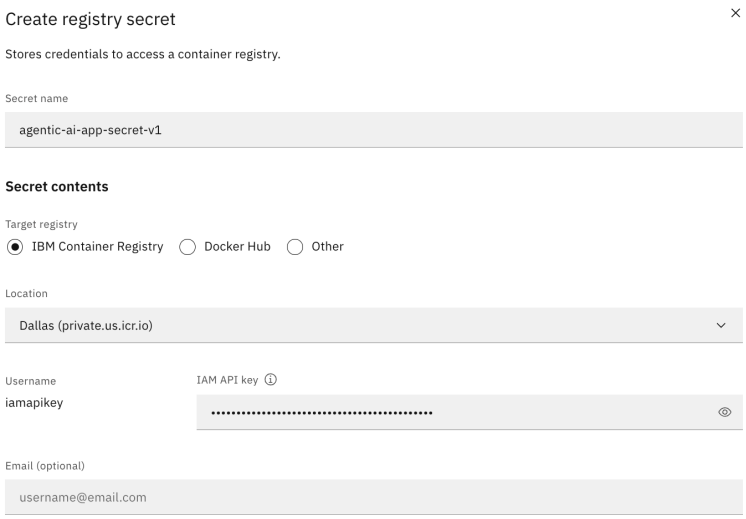
Mostly use defaults. But create new registry secret.





4.3. Create registry secret.

Name agentic-ai-app-secret.
You will also need to select region, enter the user's API key, and select the namespace that was created earlier.



Specify build details

Learn more about image builds.

Source | Strategy | Output

Registry server

private.us.icr.io

Select registry or type in registry hostname

Registry secret

agentic-ai-app-secret

Select existing or create new registry secret

Namespace

agentic-ai-app-namespace

Select or type in namespace name

Repository (image name)

Select or type in repository name

Tag

latest

Select or type in version tag

4.4. Once done with Specify build details, this is what the Code section will look.

Code

Specify a container image or build one from source code first. To learn more, see the documentation.

Use an existing container image

Build container image from source code

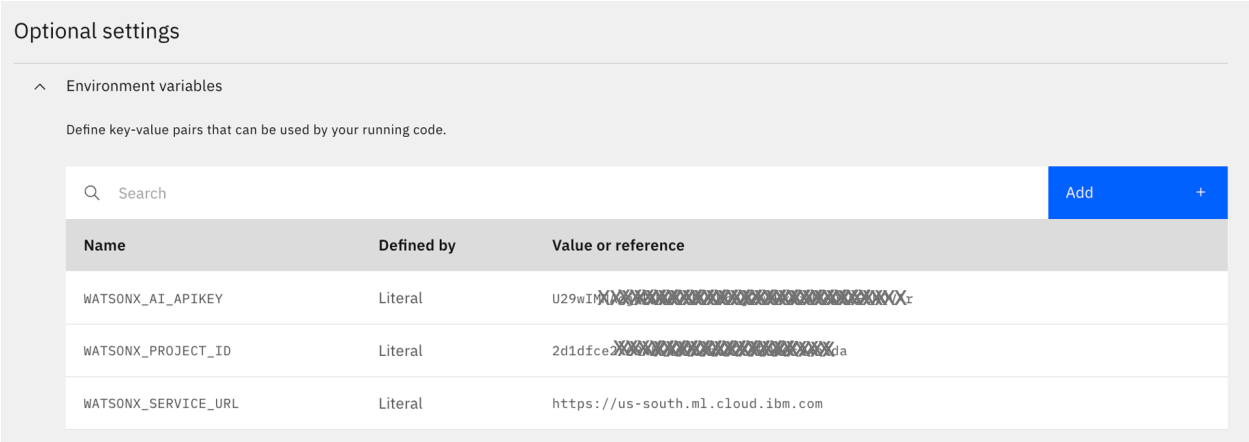
Image build details

Edit build details

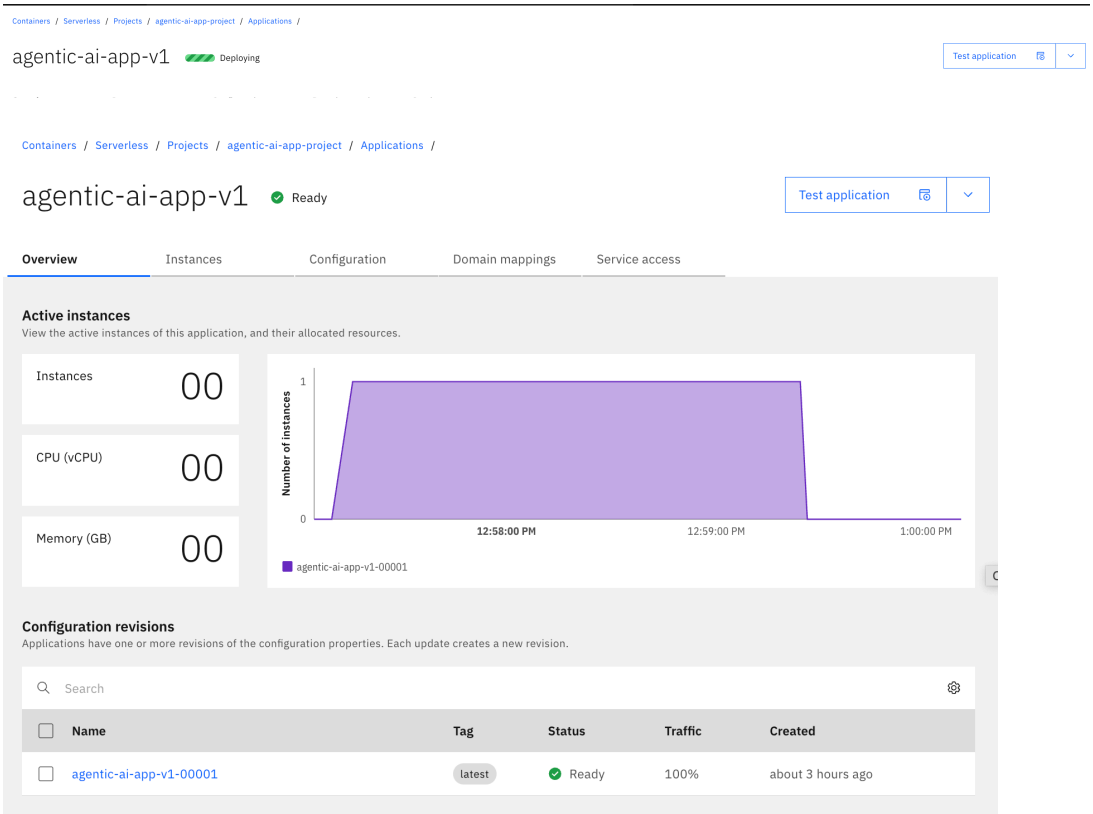
Image reference	private.us.icr.io/agentic-ai-app-namespace/agentic-ai-app-image
Registry access secret	agentic-ai-app-secret-v1
Source code URL	https://github.com/IBM/ai-agent-for-loan-risk.git
Branch name	
Context directory	/

4.5. Scroll down to optional settings and add environment variables.

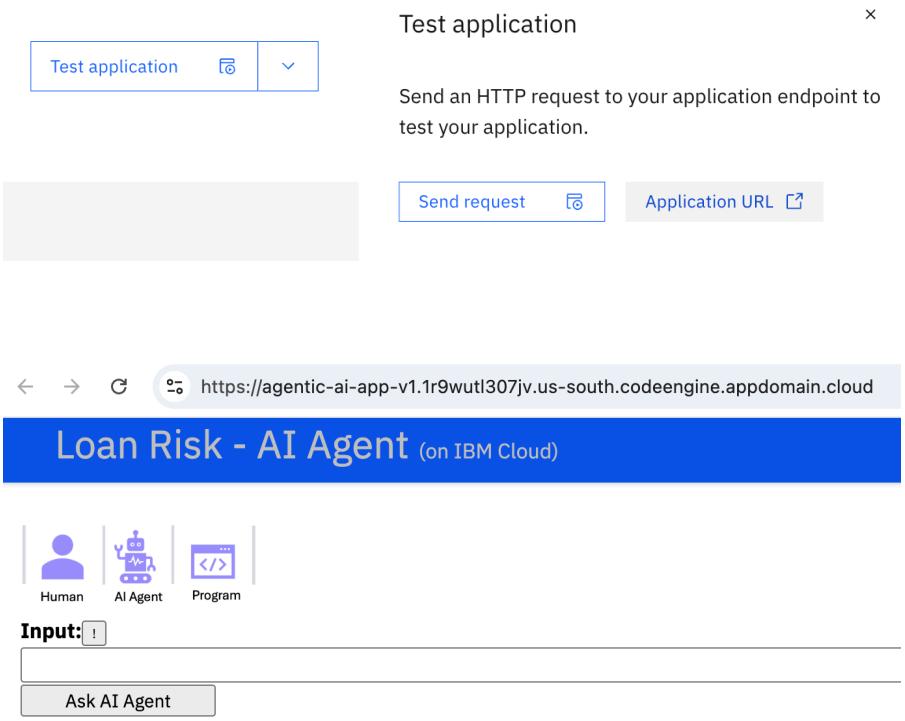
- WATSONX_AI_APIKEY
- WATSONX_PROJECT_ID
- WATSONX_SERVICE_URL



4.6. Create/deploy the application. Check status



4.7. Click on Test application (right top) > Application URL to launch application.



5. Validate the application.

Ask “What is the interest rate for matt? Explain how it was determined?”. You should get a response explain high risk and 8% interest rate.

Refer to the application usage section.

6. Enhance application with additional optional features.

Once initial application is deployed and running successfully, you can add enhancements.

- Using RAG LLM (Agentic RAG feature)
- Using watsonx Assistant/Orchestrate (Chat widget)

Refer to Optional Features section.

Optional Features

Once initial application is deployed and running successfully, you can enhance with some additional optional features.

- Using RAG LLM (Agentic RAG feature)
- Using watsonx Assistant/Orchestrate (Chat widget)

Using RAG LLM (Agentic RAG feature):

By default, the risk and interest rate tools of the AI agent simulate risk and interest rate determination. By adding this feature, the AI agent tool will make RAG query to retrieve relevant content from the bank documents and use that to determine the risk and interest rate. In the AI agent response you will be able to see the content from the document table and its interpretation by the AI agent.

- Create a new watsonx.ai Deployment Space with Deployment stage set as Production. (refer)
- Create a vector index asset in the watsonx.ai Project using the content PDF documents provided here. To create the index use vector store - "In memory", embedding model - "allminilm-l6-v2", chunksize - "2000", chunk overlap - "200".
- Open the vector index you just created in Prompt Lab and set the generative AI model to "mistral-large". Test the index by asking some questions e.g., what is the risk for credit score 655 and account status closed?, what is the interest rate for medium risk?, and confirm answers are using RAG from the content PDF documents.
- Deploy the vector index as watsonx.ai Deployment on AI service for inferencing using the "fast path" option (use the "Deploy" button). (refer-1) (refer-2) and (refer-3).
- Capture the watsonx.ai Deployment private endpoint for the vector index for RAG inferencing (use non-stream; ends with ai_service?version=...)
- On Code Engine add the following environment variables with the values captured above and redeploy the application (ENABLE_RAG_LLM=true and WATSONX_RISK_RAG_LLM_ENDPOINT=endpoint captured above)
- The application will now use the RAG content for risk and interest tools.

Using watsonx Assistant/Orchestrate (Chat widget):

By adding this feature, you can get a more conversational/chat experience when asking questions in the watsonx Assistant chat widget. The conversation is single turn and the watsonx Assistant skills can be enhanced further if needed.

- Note the Code Engine URL for the deployed application.

- Open API file (agentic-ai-app-custom-ext-openapi.json) and update the URL with the deployed application URL.
- Create an action skill in watsonx Assistant instance
- Create and add a custom extension by importing the updated Open API file (agentic-ai-app-custom-ext-openapi.json).
- Zip the folder and import the resulting zip file to set up the actions that use the custom extension (wx-asst-agentic-ai-app).
- Open the watsonx Assistant Web chat configuration and note the integrationID, region and serviceInstanceID from the Embed script tab.
- On Code Engine open the deployed application configuration, add the following environment variables with the values captured above and redeploy the application (ENABLE_WXASST=true, WXASST_INTEGRATION_ID, WXASST_REGION, WXASST_SERVICE_INSTANCE_ID captured above)
- The watsonx Assistant will become available on the page <application-url>/wx.html